

# **RBAC & Security**

Controlling Access and Hardening Your Cluster

Enterprise EKS Platform — Module 7 of 13





## Module Agenda

- Kubernetes security model — authentication, authorization, admission
- RBAC deep dive — Roles, ClusterRoles, Bindings
- ServiceAccounts and pod identity
- Security contexts at pod and container level
- Pod Security Standards and admission controllers
- Security best practices



# Kubernetes Security Model

Authentication, authorization, and admission control

# API Request Flow — Three Security Gates

Every request to the API server passes through:

## 1. Authentication

- Who are you?  
Validates identity
- Returns user/group info

## 2. Authorization

- What can you do?
- RBAC evaluates; allow or deny

## 3. Admission

- Should this be allowed?
- Mutating + validating webhooks

# Authentication Methods

## **X.509 Client Certificates**

- CN = username, O = group; used by cluster components

## **OpenID Connect (OIDC)**

- Enterprise SSO via Azure AD, Okta, Dex

## **Bearer Tokens**

- ServiceAccount tokens (JWT) and bootstrap tokens

## **AWS IAM Authenticator (EKS)**

- Maps IAM roles to K8s users via aws-auth ConfigMap

# **RBAC Deep Dive**

Role-Based Access Control in Kubernetes



# RBAC API Objects

## Define Permissions

- **Role** — namespace-scoped permissions
- **ClusterRole** — cluster-wide permissions

## Grant Permissions

- **RoleBinding** — binds Role/ClusterRole to subjects in a namespace
- **ClusterRoleBinding** — binds ClusterRole to subjects cluster-wide

**Key insight:** Roles define *what* is allowed. Bindings define *who* gets those permissions. You always need both.

# Role vs ClusterRole

## Role (Namespace)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: development
  name: pod-reader
rules:
- apiGroups: [""]          # core API group
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
- apiGroups: [""]
  resources: ["pods/log"]
  verbs: ["get"]
```

## ClusterRole (Cluster-Wide)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-viewer
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "watch", "list"]
- apiGroups: ["metrics.k8s.io"]
  resources: ["nodes"]
  verbs: ["get", "list"]
- nonResourceURLs: ["/healthz", "/readyz"]
  verbs: ["get"]
```



# RoleBinding vs ClusterRoleBinding

## RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-dev
  namespace: development
subjects:
- { kind: User, name: jane, apiGroup: rbac.authorization.k8s.io
- { kind: Group, name: dev-team, apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

## ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-node-viewers
subjects:
- kind: Group
  name: ops-team
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: monitoring-sa
  namespace: monitoring
roleRef:
  kind: ClusterRole
  name: node-viewer
  apiGroup: rbac.authorization.k8s.io
```

**Subjects** can be Users, Groups, or ServiceAccounts. **roleRef** can point to a Role or ClusterRole. ClusterRoleBindings grant permissions across ALL namespaces -- use sparingly.



# Verbs and Resources Matrix

| Verb   | HTTP Method      | Description                  |
|--------|------------------|------------------------------|
| get    | GET (single)     | Retrieve a specific resource |
| list   | GET (collection) | List resources               |
| watch  | GET (watch)      | Stream changes to resources  |
| create | POST             | Create a new resource        |
| update | PUT              | Replace entire resource      |
| patch  | PATCH            | Modify part of a resource    |
| delete | DELETE           | Remove a resource            |

**Wildcard:** Use "\*" for all verbs or all resources — avoid this outside of testing.

# Aggregated ClusterRoles

```
kind: ClusterRole
metadata:
  name: monitoring-endpoints
  labels:
    rbac.example.com/aggregate-to-monitoring: "true"
rules:
- apiGroups: [""]
  resources: ["services", "endpoints", "pods"]
  verbs: ["get", "list", "watch"]
```

```
kind: ClusterRole
metadata: { name: monitoring }
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
      rbac.example.com/aggregate-to-monitoring: "true"
rules: [] # auto-filled by controller
```

## How it works

The controller manager automatically combines rules from all ClusterRoles matching the label selector. Built-in roles like admin, edit, and view use aggregation.



# ServiceAccounts

Identity for workloads running in the cluster



# ServiceAccount Anatomy

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
  annotations:
    eks.amazonaws.com/role-arn: >-
    arn:aws:iam::123456789012:role/app-role
automountServiceAccountToken: false
```

## Key Properties

- Namespace-scoped identity
- Every namespace has a default SA
- Tokens are projected as volumes
- Annotations enable cloud integrations
- Can disable auto-mount for security

# \$ ServiceAccount Token Projection

```
spec:
  serviceAccountName: app-sa
  volumes:
  - name: sa-token
    projected:
      sources:
      - serviceAccountToken:
          path: app-token
          expirationSeconds: 3600
          audience: api
# ... container volumeMounts reference /var/run/secrets/tokens
```



## Binding Roles to ServiceAccounts

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-sa-pod-reader
  namespace: production
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

**Pattern:** Create a dedicated ServiceAccount per application, grant only the permissions it needs, and reference it in the pod spec with `serviceAccountName`.

# Pod Identity — IRSA on EKS

## **IAM Roles for Service Accounts**

- Maps K8s ServiceAccount to IAM Role
- Uses OIDC federation
- No shared node-level credentials
- Fine-grained per-pod access

## **How IRSA Works**

1. EKS cluster has an OIDC provider
2. IAM role trusts the OIDC provider
3. SA annotated with IAM role ARN
4. Pod receives projected OIDC token
5. AWS SDK exchanges token for creds





# Security Context

Constraining what pods and containers can do



# Pod SecurityContext

```
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  # ... containers follow
```

**runAsUser/runAsGroup:** Sets the UID/GID for all containers in the pod

**fsGroup:** Sets the group ownership of all mounted volumes



# Container SecurityContext

```
containers:  
- name: app  
  image: my-app:1.0  
  securityContext:  
    allowPrivilegeEscalation: false  
    readOnlyRootFilesystem: true  
    capabilities:  
      drop: ["ALL"]  
      add: ["NET_BIND_SERVICE"]  
    privileged: false
```

## Container-level overrides pod-level settings

- **allowPrivilegeEscalation:**  
Prevents gaining more privileges than parent
- **readOnlyRootFilesystem:**  
Prevents writes to container filesystem
- **capabilities:** Fine-grained Linux capability control
- **privileged:** Never set to true for application workloads



# Building Secure Container Images

## Minimal Images

- **Multi-stage builds** — compile in one stage, copy only the binary to the final image
- **Distroless** — Google's images with no shell or package manager
- **Alpine** — ~5 MB base; good for interpreted languages
- **scratch** — empty base for statically linked Go / Rust binaries

```
# Multi-stage build example
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app

FROM gcr.io/distroless/static:nonroot
COPY --from=build /app /app
USER 65532:65532
ENTRYPOINT ["/app"]
```

**Result:** Final image is ~15 MB with no shell, no root, no package manager

# Image Security Pipeline

## Run as Non-Root

- Add USER in Dockerfile
- Pair with `runAsNonRoot: true`
- Never store secrets in layers

## Image Scanning

- **Trivy** — CI-friendly scanner
- **ECR scanning** — on push or continuous
- Scan in CI *and* in registry

## Image Signing

- **Cosign** — keyless OIDC signing
- Verify at admission
- Pin by `@sha256:` digest

**Supply-chain rule of thumb:** Build minimal → scan for CVEs → sign the artifact → verify at deploy

# Pod Security Standards

## Privileged

- Unrestricted; all capabilities allowed
- For CNI plugins, storage drivers only

## Baseline

- Prevents known escalations
- Good starting point for migration

## Restricted

- Non-root, drop all capabilities
- Recommended hardening target

**Recommendation:** Target **Restricted** for application workloads. Use **Baseline** as a transition step. Reserve **Privileged** for infrastructure components only.

## Enforcing via Namespace Labels

```
apiVersion: v1  
kind: Namespace
```

# **Security Best Practices**

Hardening your cluster



# Principle of Least Privilege in Practice

## RBAC Least Privilege

- Use Roles over ClusterRoles when possible
- Grant specific verbs — avoid \*
- Name specific resources
- Use resourceNames for fine-grained control
- Audit with `kubectl auth can-i`

```
# Fine-grained: specific resource names
kind: Role
metadata: { namespace: production, name: configmap-updater }
rules:
- apiGroups: ["" ]
  resources: ["configmaps"]
  resourceNames: ["app-config"]
  verbs: ["get", "update"]
```

```
# Verify permissions
kubectl auth can-i get pods --as=jane -n dev
kubectl auth can-i --list \
  --as=system:serviceaccount:prod:app-sa
```





# Audit Logging

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  resources:
  - { group: "", resources: ["secrets"] }
- level: RequestResponse
  resources:
  - { group: "rbac.authorization.k8s.io",
      resources: ["clusterroles", "clusterrolebindings"] }
- level: None
  resources:
  - { group: "", resources: ["endpoints", "services/status"] }
```

## Audit Levels

- **None:** Skip logging
- **Metadata:** Request metadata only
- **Request:** Metadata + request body
- **RequestResponse:** Log everything

# Image Security

## Image Scanning

- Scan in CI/CD pipeline (Trivy, Grype)
- ECR image scanning (on push)
- Block critical CVEs from deploying
- Scan base images regularly

**Always use:**  
`imagePullPolicy: Always`  
with immutable tags or digests

## Image Signing & Verification

- Sign images with Cosign / Notation
- Verify signatures at admission
- Use allowlisted registries only
- Pin images by SHA256 digest

```
# Pin image by digest
containers:
- name: app
  image: 123456789012.dkr.ecr.us-east-2
    .amazonaws.com/app@sha256:abc123...
```

# EKS Security Best Practices

## Cluster Configuration

- Enable envelope encryption for secrets
- Use private API endpoint
- Enable control plane logging
- Restrict CIDR for API access
- Keep Kubernetes version current

## Workload Configuration

- Use IRSA or EKS Pod Identity
- Enforce Pod Security Standards
- Use NetworkPolicies (needs CNI support)
- Rotate ServiceAccount tokens
- Use external secrets (Secrets Manager)

**aws-auth ConfigMap:** Regularly audit this to ensure only authorized IAM principals have cluster access. Consider migrating to EKS access entries for more granular control.



# IRSA: IAM Roles for Service Accounts

Least-privilege AWS access for pods

# ! Why Node-Level IAM Is Dangerous

**Without IRSA:** Every pod on a node inherits the EC2 instance profile — the same IAM role for all workloads.

- **Over-privileged pods** — a logging sidecar gets the same S3/SQS access as your data pipeline
- **Blast radius** — one compromised container can access every AWS resource the node role permits
- **No audit granularity** — CloudTrail shows the node role, not which pod made the API call

**IRSA solves this** by mapping each Kubernetes ServiceAccount to its own IAM role with scoped permissions — true least privilege for AWS access.

# How IRSA Works: OIDC → STS → Credentials

## End-to-End Flow

1. **EKS cluster exposes an OIDC endpoint** — registered as an IAM Identity Provider
2. **Pod starts** — EKS webhook injects a projected JWT token + `AWS_ROLE_ARN` env var
3. **AWS SDK calls STS** `AssumeRoleWithWebIdentity` with the JWT
4. **STS validates** the token signature via the OIDC provider's JWKS
5. **STS returns temporary credentials** scoped to that ServiceAccount's IAM role

**Zero code changes:** The AWS SDK credential chain detects IRSA environment variables automatically

# Configuring IRSA

## 1. Annotate the ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: my-namespace
  annotations:
    eks.amazonaws.com/role-arn: >-
      arn:aws:iam::111122223333:role/
      my-app-irsa-role
```

The annotation tells the EKS webhook which IAM role to inject into pods using this SA.

## 2. IAM Trust Policy

```
{
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated":
      "arn:aws:iam::111122223333:oidc-provider/oidc.eks..." },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": { "StringEquals": {
      "OIDC_ID:aud": "sts.amazonaws.com",
      "OIDC_ID:sub": "system:serviceaccount:my-namespace:my-app-
    }}
  ]
}
```

**Security boundary:** The `:sub` condition scopes the role to an exact namespace + ServiceAccount. Without it, any SA in the cluster could assume this role.

# **IRSA Best Practices**

## **Do**

- One IAM role per workload
- Scope policies to specific resource ARNs
- Always include `:sub` condition in trust policies
- Verify with `aws sts get-caller-identity`

## **Avoid**

- Sharing the node instance profile
- Wildcard actions or `"Resource": "*"`
- Broad `StringLike` on `:sub`
- Long-lived AWS keys in Secrets

**EKS Pod Identity** (GA 2023) simplifies IRSA by removing trust-policy management. IRSA remains the standard for existing clusters.



# Lab 7: RBAC and Security

## Hands-on: Securing your cluster workloads

- Create Roles and RoleBindings for namespace-scoped access
- Test permissions with `kubectl auth can-i --as`
- Create dedicated ServiceAccounts for applications
- Apply security contexts to pods and containers
- Configure Pod Security Admission on a namespace
- Verify that non-compliant pods are rejected
- Annotate a ServiceAccount for IRSA and verify scoped AWS credentials

**Duration:** ~45-55 minutes

# Key Takeaways

**Least Privilege:** Grant minimum permissions needed. Roles define what; Bindings define who. Start with no access and add as required.

**PSS Restricted Profile:** Use the restricted Pod Security Standard to enforce non-root, read-only FS, and no privilege escalation. Combine with securityContext and audit logging for defense in depth.

**IRSA for Pod Identity:** Per-pod AWS credentials via OIDC federation eliminate shared node IAM roles and static credentials.

**Audit Logging:** Enable audit logging and forward to your SIEM for detecting unauthorized access and policy violations.

# ? Questions & Discussion

How does your team currently manage access control in your Kubernetes clusters?

Up Next: Module 8 — Network Policies